

pact is reassessed based on the known existing controls, processes/procedures, etc. that reduce/mitigate the risk.

Determining if a risk is considered Low, Medium, High, or Critical

Once the Inherent and Residual risk score is determined, the following table can be used to determine if a risk is considered Low, Medium, High, or Critical:

Risk Rating	Risk Score Range
Low	1-10
Medium	11-20
High	21-25
Critical	26-30

These ratings represent labels for communication purposes rather than what is or is not acceptable. To determine what is an acceptable risk, please refer to risk tolerances.

Ad-hoc Risk Identification and Assessment

There may be times that risks are identified outside of traditional risk assessments - such as risks that arise from a security incident, risk identified through regular day-to-day business operations, etc. All security operational risks identified ad-hoc are discussed with the Security Risk Team, an inherent risk score is assigned, and a qualitative analysis done to determine if it should be escalated to the risk register.

Risk Response

For each risk identified, a formal risk response decision is made to determine how GitLab will handle the risk. As part of the risk response procedures, the Risk Owner will make a determination on whether or not to accept a risk or pursue remediation based on our Risk Appetite and Tolerances. Treatment plans will be reviewed by the Security Risk Manager or delegate and approval captured via comment in the GRC application or associated GitLab issue.

RESPONSE	OPTION	DEFINITION
Monitor (do nothing)		The risk score falls within our risk tolerance levels and no additional actions are required at this time. Risks that have been treated, resulting in a risk score that is within the risk tolerance level will be given the status of Monitor within our GRC system.
Remediate the Risk		The risk is not within our risk tolerance. Complete a risk remediation plan to remediate the risk through: Sharing the risk (identify and implement solutions to share the risk with other parties), Reducing the likelihood of occurrence, and/or Reducing the level of impact to GitLab
Accept the Risk		The risk is not within our risk tolerance. Accept or take the risk without executing a remediation plan because the cost to pursue remediation is higher than the potential benefit.

The risk object in the GRC application will be updated to reflect the agreed upon risk response. If "Remediate the Risk" is selected, the Risk Owner will execute a Risk Treatment

Plan. The documented plan and status of the risk treatment will be captured within the GRC application as well. See below for more information about risk response options.

Monitor (nothing beyond expected iteration)

In the cases where a risk owner has concluded that a risk is within tolerance, no additional action is required besides ensuring that the StORM Program DRI agrees with the treatment option.

Remediate the Risk

When choosing to remediate the risk, a specific path must be selected:

- Remediate by reducing the likelihood that the risk could occur
- Remediate by reducing the impact to GitLab if the risk occurs
- Remediate by sharing or transferring the risk with a third party
- Remediate by avoiding the risk by deciding not to start or continue with the activity that gives rise to the risk

Once a path is selected, the Risk Owner is required to provide a SMART, detailed plan that includes milestones and due dates for working towards risk remediation. The treatment plan must be achievable and address the root cause of the risk event. Additionally and in alignment with our value of Transparency, each treatment plan will include a step for documenting the results/outcome of the remediation within the Handbook. If the result of the remediation is considered not public and cannot be documented within the Handbook, it should be documented within our Internal Handbook or an internal runbook. The Security Risk Team will leverage these risk treatment plans to track the status of risk remediation.

If the risk treatment plan is executed and results in a downgrading of the residual risk level for the risk (ex. the residual risk level goes from High to Medium), validation of the remediation will be performed and captured within the associated risk object. Quality review of the downgrade support documentation will be completed by the Security Risk Manager and captured via comment in the GRC application.

Accept the Risk

In the cases where a risk owner has opted to pursue a risk acceptance, the following approvals will be required based on risk rating that was assigned to the risk undergoing a risk acceptance:

Risk Level	Approval Level Required
-----	-----
CRITICAL	Risk Owner + VP Level Approval + E-group Level Approval
HIGH/MEDIUM	Risk Owner + VP Level Approval

If the Risk Owner is a VP, no additional VP level approval is required

By accepting the risk, the Risk Owner and risk acceptance approvers (if separate from Risk Owner), agree to reassess the risk on an annual basis to determine whether risk acceptance is the best response option for the respective risk. If risk acceptance is appropriate based on

the annual assessment, approvals will be re-obtained based on the risk and approval requirements noted in the table above. Additionally, the Risk Owner will be on point for remediation in the event the risk is realized or risk acceptance is no longer appropriate.

Risk Tracking and Reporting

Identified risks are formally tracked via an internal risk register. Given the nature of the sensitivity of this information in aggregate, the risk register is not made public, and is not distributed externally. However, a publicly viewable GitLab Risk Register Template is available [here](#) for those interested in getting some more insight into the type of information tracked in GitLab's risk register. StORM-related risk activities are centralized within GitLab (internal only).

We report on our top 5 risks on a quarterly basis (the Security Risk Quarterly or "SRQ") in alignment with our values. To learn more about the SRQ, please see our YouTube unfiltered video [linked here](#). The template we've used can be found [here](#) for reference. Additionally, we perform an annual exercise to refresh our Risk Appetite and our Top 5 risks.

The StORM program is integrated within daily operations at GitLab. As such, we have mapped our StORM risks to risk sources including Security Compliance controls and observations and Product Security's Risk Register. Our mapping exercises are intended to provide a more comprehensive view of our security risk landscape and to enable StORM engineers to more quickly identify and escalate risks.

GitLab team members can find all Security Risk Quarterly documents in the SRQ Reports Google Drive folder.

Top 5 Risks

The Security Division's "Top 5 Risks" are established annually and are reported upon quarterly as resources allow via the SRQ. Security Leadership leverages these Top 5 Risks when conducting short and long-term strategic planning activities. We intend to support remediation through assisting with treatment activities and performing design and operating effectiveness assurance testing on key remediation activities.

Long-Term Risk Treatment Planning

Executive Risk Owners are accountable for ensuring that long-term treatment plans are established and executed for each of the Top 5 Risks. The Security Risk Team is responsible for coordinating long-term treatment planning with the Executive Risk Owner and the associated Risk Owner(s). The following template is leveraged during planning to ensure consistency in our approach and in reporting:

- Establish success criteria that would move the risk within tolerance. These criteria will become Key Results (KRs) and are the milestones used to gauge progress of risk remediation. Completed KRs will correlate to a reduced risk score.
- Identify dependencies for each KR.
- Identify a Risk Owner responsible for delivering each KR. This person should be in the department responsible for implementing the treatment activity.
- Establish realistic target delivery dates for each KR. Due to the nature of operational

risks, we expect delivery dates to range up to 4 years in the future. Try to establish at least one KR per quarter to show incremental progress.

- Scoring: Subtract '10' (our risk tolerance threshold based on the current risk appetite) from the current risk score to identify the reduction required to move the risk within tolerance. Divide this difference by the number of projected KRs. The quotient/remainder is the amount the score will be reduced each time a KR is completed. This number can be adjusted judgmentally as required.

In the event the Executive Risk Owner chooses not to pursue a Risk Remediation-related KR in a given quarter due to competing priorities, a Risk Acceptance should be formalized to document the business rationale. This Risk Acceptance should contain rationale explaining why the risk of delaying additional Risk Remediation is less than the risk of not fulfilling the competing priority.

StORM Reporting Schedule

The table below outlines planned/completed activities for FY25.

Timing	Activities
----- -----	
Q1	SRQ
Q2	SRQ and Annual Refresh Planning
Q3	SRQ with refreshed risk appetite and Top 5 risks
Q4	SRQ

Exceptions

The only exceptions to this procedure are those risks that are out of scope (as defined above).

References

- GitLab Handbook References:
 - GitLab's Communication Page, Not Public Section
- External References
 - NIST SP 800-30 Rev. 1
 - ISO 31000 Risk Management Methodology

SECTION: security/security-assurance/security-risk/storm-program/business-impact-analysis.md

Purpose

The Business Impact Analysis (BIA) helps determine the systems critical to serving GitLab's Customers. The output of the BIA is the designation of a Critical System Tier (CST) for a new system. The CST aids Security Compliance in Scoping and is also available to help business owners determine the prioritization of system restoration efforts in the event of a disruption.

Note: In adjacent to the BIA, questions may be asked to satisfy global privacy regulation requirements pertaining to a system's Personal Data processing.

Scope

The BIA procedure covers all new systems to be added to GitLab's Tech Stack.

Roles and Responsibilities

Role	Responsibility
Security Risk Team	Responsible for executing and maintaining this procedure.
Business/Technical Owner of a System	Responsible for answering BIA questions during procurement and validating CST designation proposed by Security Risk.
Security Assurance Management (Code Owners)	Responsible for approving significant changes and exceptions to the BIA.

BIA Procedures

New Systems

All requisitions in Zip that involve the use of a new system to collect, store, or transmit GitLab data require a BIA.

When creating this type of requisition, the Business/Technical Owner is prompted to answer two BIA questions at minimum:

1. What is the impact of System disruption (System is unavailable)?
2. Please describe the potential impact of System disruption. Specify any GitLab team(s) affected.

The answers to these questions help the Security Risk Team designate a Critical System Tier for the new system during the TPRM Assessment process. Additional questions may be asked by Security Risk to validate the assignment of an appropriate CST.

Quality Reviews

The output of the BIA (designation of a Critical System Tier for a new system) is peer-reviewed by a member of Security Risk at the conclusion of the TPRM Assessment.

Reporting

The output of the BIA is reported when adding a new system to GitLab's Tech Stack. The data field for reporting is: criticalsystemtier.

Exceptions

System Proof of Concepts (POC), Proof of Values (POV), and Pilots are exempt from BIA procedures.

References

- Critical System Tiering Methodology

- Data Classification Standard

SECTION: security/security-assurance/security-risk/storm-program/critical-systems.md

Purpose

The purpose of the Critical System Tiering Methodology is to support GitLab in categorizing systems based on their impact on GitLab's SaaS subscriptions (delivery of GitLab.com or GitLab Dedicated). By classifying systems into specific tiers, GitLab may appropriately prioritize risk mitigation activities and tailor internal controls.

Scope

The Critical System Tiering methodology is applicable to all systems utilized across GitLab which are tracked within the Tech Stack.

Roles and Responsibilities

Role	Responsibility
------	----------------

-----	-----
-------	-------

Security Risk Team	Owns the Critical System Tiering Methodology and designates Critical System Tiers for new systems through the Business Impact Analysis.
--------------------	---

IT Compliance	Supports defining of Critical System Tiers in conjunction with the Security Risk Team when new systems are added to the Tech Stack.
---------------	---

Business/Technical Owners of Systems	Provide complete and accurate data about the systems that they own so that an accurate tier is designated.
--------------------------------------	--

Critical System Tiering Procedure

Defining what a critical system means at GitLab can be complex given the nature of our environment and the number of integrations that exist across the many systems that are used to carry out business activities. GitLab's Business Impact Analysis (BIA) procedure enables the designation of a Critical System Tier for a new system by the Security Risk Team. The BIA questions used to designate Critical System Tier are:

1. What is the impact of System disruption (System is unavailable)?
2. Please describe the potential impact of System disruption. Specify any GitLab team(s) affected.

The Security Risk Team judgmentally designates a Critical System Tier for the new system based on answers from the Business Owner and their own understanding of the services provided by the system. The guidelines used to distinguish each tier are described in the next section.

Designating Critical System Tiers

Systems are designated a Critical System Tier based on the following matrix:

<style type="text/css">

```

.tg {border-collapse:collapse;border-spacing:0;margin:0px auto;}
.tg td{border-color:black;border-style:solid;border-width:1px;overflow:hidden;padding:10px
5px;word-break:normal;}
.tg th{border-color:black;border-style:solid;border-width:1px;overflow:hidden;padding:10px
5px;word-break:normal;}
.tg .tg-zqun{background-color:ffffff;color:000000;text-align:center;vertical-align:middle}
.tg .tg-knp3{background-color:6e49cb;border-color:000000;color:ffffff !important;;
text-align:center;vertical-align:middle}
.tg .tg-clye{background-color:380d75;color:ffffff;font-weight:bold;text-align:center;vertical-
align:middle}
.tg .tg-fecx{background-color:cccccc;color:000000;font-weight:bold;text-align:center;vertical-
align:middle}
.tg .tg-cc97{background-color:380d75;color:ffffff;text-align:center;vertical-align:middle}
.tg .tg-dxvi{background-color:6e49cb;color:ffffff;font-weight:bold;text-align:center;vertical-
align:middle}
.tg .tg-e02t{background-color:ffffff;border-color:000000;color:000000 !important;;
font-weight:bold;text-align:center;vertical-align:middle}
.tg .tg-9hzb{background-color:FFF;font-weight:bold;text-align:center;vertical-align:top}
</style>
<table class="tg">
<tbody>
<tr>
<td class="tg-clye">Critical System Tier (CST) <span style="color:DB3B21;"></span></td>
<td class="tg-dxvi">CST Description</td>
<td class="tg-dxvi">Examples</td>
</tr>
<tr>
<td class="tg-e02t">Tier 1 Mission Critical<span style="color:DB3B21;"></span></td>
<td class="tg-zqun">Disruption or breach has an immediate and significant impact on the
availability/security of GitLab SaaS subscriptions and Customer data (See <a
href="/handbook/security/standards/data-classification-standard/data-classification-
definitions">Data Classification Standard</a> for definitions).</td>
<td class="tg-zqun">GitLab.com, Google Cloud Platform, Devo</td>
</tr>
<tr>
<td class="tg-e02t">Tier 2 Business Critical<span style="color:DB3B21;"></span></td>
<td class="tg-zqun">Disruption affects execution of major business functions or
capabilities.</td>
<td class="tg-zqun">Okta, Salesforce, Workday</td>
</tr>
<tr>
<td class="tg-e02t">Tier 3 Business Operational</td>
<td class="tg-zqun">Disruption affects efficiency or cost in completing standard business
processes.</td>
<td class="tg-zqun">DocuSign, Figma, Tableau</td>
</tr>
<tr>
<td class="tg-e02t">Tier 4 Administrative</td>
<td class="tg-zqun">Disruption affects productivity/well-being of individual GitLab Team
Members.</td>

```

```

        <td class="tg-zqun">Clockwise, Donut</td>
    </tr>
</tbody>
</table>
<br/>

```

```

{{% panel header="Notes" header-bg="primary" %}}
{{% note %}}
<span style="color:DB3B21;"><b>\</b></span> As an extension of tiering methodology, the Data
Classification Standard prescribes specific Security and Privacy control requirements for each
data classification level. These requirements should be followed based on a system's data
classification, regardless of the system's tier.
{{% /note %}}
{{% note %}}
<span style="color:DB3B21;"><b>\</b></span> By default, any system that contains <b>RED
Data</b> per the Data Classification Standard OR is a Third Party Sub-Processor will be a Tier
1 Mission Critical system. This is due to the fact that this data is Customer-owned and
uploaded and as such, has been deemed to be mission critical in nature.
{{% /note %}}
{{% note %}}
<span style="color:DB3B21;"><b>\</b></span> By default, any system in-scope for SOX will be a
Tier 2 Business Critical system, at minimum.
{{% /note %}}
{{% /panel %}}

```

```

{{% panel header="Why do we differentiate between impact on SaaS subscriptions vs. major
business functions or capabilities?" header-bg="warning" %}}
Critical System Tiers help prioritize risk mitigation activities for systems critical to
serving GitLab's Customers. GitLab's core service offerings are SaaS. Thus, the systems that
have an immediate and significant impact on the availability/security of GitLab SaaS
subscriptions and Customer data in the event of a disruption are priority. These are Tier 1
Mission Critical systems.
{{% /panel %}}

```

Why does GitLab need this methodology?

Tiering systems utilized across GitLab enables team members to make decisions on prioritizing work related to a specific system(s). For example, if multiple risks have been identified across many systems and require remediation, team members can leverage the designated Critical System Tiers to make decisions on how to prioritize remediation efforts. This methodology also provides GitLab with a mechanism to easily identify systems which are most critical across the organization so that teams can proactively protect and secure those systems.

Maintaining Critical System Tiers

Critical System Tiers for existing systems are validated periodically. A system's designated tier can be found in the techstack.yml file which is the Single Source of Truth for all systems used at GitLab.

Exceptions

Exceptions to this procedure will be tracked as per the Information Security Policy Exception Management Process.

References

- Business Impact Analysis
- Data Classification Standard

SECTION: security/security-operations/_index.md

Vision

Protect company property by identifying, preventing, detecting and responding to risks and security events targeting the business and GitLab.com and its users. We are at the forefront of GitLab's security.

Mission

The Security Operations department focuses on the operational aspect of security. Our department consists of experienced breakers, builders, and defenders from all walks of life and geographic locations. We are responsible for improving GitLab's security capabilities and metrics in the areas of security anomaly/event detection and incident response and abuse of GitLab.com.

Our department consists of:

- Security Incident Response Team (SIRT) - Security detection engineering and incident response
- Trust & Safety - Prevention and mitigation of abuse of the GitLab.com platform
- Red Team - Adversary emulation
- Threat Intelligence - Actionable insights on real-world threats
- Signals Engineering - Detection Engineering

Contacting Security Operations

- Slack channel to interact with the Security Operations Department security-operations
- The Security Operations department follows the Security Team On-Call Rotation with more details available in Security Operations On-Call
- How to Engage the Security Engineer On-Call

Common Links

- GitLab Security Division Group (GitLab)
- GitLab Security Operations Department Group (GitLab)

SECTION: security/security-operations/secops-oncall.md

The Security Operations sub-department is collectively responsible for responding to reports of actual or potential security incidents on a 24/7/365 basis.

- The SIRT (Security Incident Response Team) generally responds to reports of suspicious activities (e.g. Phishing, hacking, social engineering) and security alerts.
- The Trust & Safety team generally responds to reports of cryptomining, platform SPAM, abuse, bullying and violations of terms of service.

Prior to scheduling planned time off, Security Operations team members should consult with the team to ensure proper coverage will be available.

Security Operations Managers also share in On-Call responsibilities and need to ensure proper coverage for escalations and emergencies. The Security department maintains a series of On-Call escalations to ensure that every reported incident is responded to in a reasonable timeframe.

On-Call scheduling for SIRT is organized in Pager Duty within the Security Responder policy.

On-Call scheduling for Trust & Safety is organized in Pager Duty within the Trust & Safety Responder policy.

On-Call Security Handoffs

Times

Standard handoff times are as scheduled. However, team members are empowered to agree on a temporary modified handoff schedule as long as all those involved agree and the team is notified about changes.

SIRT (November - April)

- 23:00 to 07:00 UTC -> APAC - 8 hours
- 07:00 to 15:00 UTC -> EMEA - 8 hours
- 15:00 to 23:00 UTC -> AMER - 8 hours

SIRT (April - November)

- 22:00 to 06:00 UTC -> APAC - 8 hours
- 06:00 to 14:00 UTC -> EMEA - 8 hours
- 14:00 to 22:00 UTC -> AMER - 8 hours

SIRT times are reflected in the SIRT Handoffs calendar. In case both parties agree to change the time, this should be reflected in the calendar at least 24 hours prior to the handoff. This allows other team members the ability to adequately plan their schedules the night before. Changing the meeting time counts as a notification.

Trust and Safety

- 23:00 to 07:00 UTC -> APAC - 8 hours
- 07:00 to 15:00 UTC -> EMEA - 8 hours
- 15:00 to 23:00 UTC -> AMER - 8 hours

SIRT Written Handoff

Incident issues are the SSoT for any incident. Be sure to include any significant incident updates within the incident summaries.

SIRT is using the self-developed tool Handogotchi (check promotion of Handogotchi on Tine) for written handover summaries. Handogotchi reminds the SIRT engineer on call to update incidents and add additional information one hour before handoff time. It will automatically send links to open incidents half an hour before handoff times.

Written handoffs are required to be completed at least half an hour before the end of every shift and are the basis for warm handoffs.

For all incidents, we highly encourage pausing after no longer than one hour to document and make it a general habit to keep documentation up to date after every work step. For all questions on documentation, please consult the Incident Report Writing Guide

SIRT Warm Handoff

SIRT uses warm handoffs to clarify written handoffs and avoid misunderstandings in complex situations. They should take no more than 15 minutes. One person per region is required (outgoing and incoming).

The outgoing region prepares the handoff as described in the section above. The incoming region should familiarize themselves with the written handoff before attending the meeting. Team members should be prepared for warm handoff meetings, so that the meetings will be efficient, with a focus on open questions and clarifications.

Because the incidents are well documented, there is no written agenda for warm handoffs. All significant discussion points that come up must be immediately documented and corrected in the incident issue.

Warm handoffs are only required during the week. This is because warm handoffs would otherwise have to be performed at around midnight on weekends for EMEA.

SIRT On-Call

During all On-Call shifts, the On-Call Engineer is filling the role of the "Triage Engineer".

Weekday

Weekday On-Call Engineer Responsibilities

Security Operations provides weekday On-Call coverage using a follow-the-sun model. Weekday On-Call Security Engineers are the team members that cover the On-Call responsibilities during their region's sunny time.

- A Weekday On-Call shift typically covers one work week, from Monday to Friday.
- The Weekday On-Call Security Engineer is expected to cover the "Triage Engineer" role, and should be the first responder to alerts and incidents as they are presented via PagerDuty and

via the various Slack and alerting channels.

Weekend

Weekend On-Call Security Responsibilities

The Weekend On-Call Security team member will be responsible for covering On-Call responsibilities from AMER Friday evening until APAC Monday morning according to the established On-Call Security Handoff times.

- The Weekend On-Call Security team member is responsible for timely responding to PagerDuty and the incident response process in accordance with the "Triage Engineer" role.
- During the weekend hours, the Weekend On-Call Security team member is not expected to be at their computer working unless they are responding to an incident, triaging alerts, or working on a leadership request for critical deliverables. However, they will be asked to acknowledge any Pager Duty incidents via their mobile phone within 15 minutes, and begin triaging the incident within one hour. Priority 3 Severity 3 and below incidents can be handled by the team as weekday tasks.
- The Weekend On-Call is responsible to acknowledge, and triage all alerts once a day. Resulting incidents should be either worked or delegated according to the triage guidelines. See "Alert Triage" for details.
- When there is a significant business need, the Weekend On-Call Security team member may be responsible for ensuring continuity of critical deliverables as determined by leadership. Critical deliverables will generally consist of, but are not limited to: threat hunting and mitigation, bug fixes, cryptomining, customer impacting SPAM, or detection and response engineering to protect against credible threats.

Weekend On-Call Security Scheduling

- Weekend On-Call slots will be filled on a volunteer basis, meaning team members can sign up for the slots that would fit their schedules. The SIRT and Trust and Safety teams will have individual Weekend On-Call schedules so that one person from each team will cover the Weekend On-Call shift. General guidance is that team members should volunteer for the appropriate ratio of qualified team members to weekends requiring coverage. For example, in a team of 12 qualified team members, volunteering for 4 to 5 weekends per year, per person would allow for full coverage. Please note, ratios may change from time to time depending on the size of the team.
- If there are open remaining slots, the Security Operations managers will assign slots to team members who have not hit their target amount of slots. For example, if there's an open slot and a team member that should volunteer for 4 to 5 slots but has signed up for 2, the Manager can allocate the slot to that team member. If the team member is unavailable, they would be responsible for finding a replacement.
- In case of emergencies, managers may seek other volunteers and in rare cases may need to assign coverage.

Weekend On-Call Security Relief

- Current On-Call metrics show an average of less than one weekend page per month.
- Because busy On-Call weekends sometimes happen, and because the weekend on-call team member needs to triage alerts, in recognition of weekend coverage, GitLab is proactively asking team

members to take the next two business days off following their Weekend On-Call shift as time off in lieu, regardless of whether the team member had to respond to an incident.

When scheduled for the Weekend On-Call Security shift, team members should:

- Submit time off in lieu for the next two business days immediately following their Weekend On-Call shift. Because this time off is intended to provide team members with immediate relief, these days cannot be moved to days later in the week or month, regardless of other planned or unplanned time off.
- If a holiday or Friends and Family day falls on the next business day, time off in lieu can be taken for the subsequent two days.
- Commit to covering the Weekend On-Call shift. Avoid signing up for weekends you are unable to cover, or otherwise have a conflict.
- Find replacement coverage if you cannot commit to covering the entire weekend.

SIRT On-Call Paging

On-Call Paging Workflow

The SIRT On-Call paging workflow is currently designed to follow this escalation path:

1. The first notification goes to our incident slack channel.
1. The designated Security Engineer On-Call in the sunny region is paged after 5 minutes of no response.
1. All Security Engineers in the sunny region are paged after 10 minutes of no response. During the weekend, one person will have volunteered for sole responsibility of weekend coverage. Team members not designated as the Security Engineer On-Call can and should provide assistance if the Security Engineer On-Call misses the page. When weekend assistance is provided, team members should request taking time off in lieu with their manager, and target 1:1 (hour for hour) time off, immediately following the weekend as long as proper coverage is available.
1. The Security Operations manager in the sunny region is paged as a backup after 15 minutes if the team members don't acknowledge the pages.
1. Security Managers who volunteer as backups are paged if SIRT does not acknowledge the previous pages after 15 minutes.
1. The VP of Security Operations is paged if Security Managers don't acknowledge the pages after 15 minutes.

SIRT On-Call Paging Duties

The On-Call Engineer's primary concern is to provide timely and adequate responses to incoming pages. When receiving a page:

1. Acknowledge the alarm in the corresponding alert Slack channel or through PagerDuty directly.
1. Review incident's GitLab issue and follow the checklists posted there for triaging.

If the alarm is not acknowledged, the Security Incident Manager On-Call will be alerted.

Engineers should acknowledge pages within the first 15 minutes, and start performing initial triage of potential incidents within the first hour of the initial page.

Security Managers On-Call

In addition to the Security Engineers being On-Call, the Security Managers across the GitLab Security Department act as backups in the event the Security Engineers are unable to acknowledge security pages. PagerDuty will automatically engage the Security Manager On-Call if SIRT doesn't acknowledge the paging attempts.

It's the responsibility of the Security Manager On-Call to:

- Be available via mobile phone during their On-Call shift if the Security Engineer On-Call does not acknowledge a page.
- Attempt to contact the Security Engineer On-Call to acknowledge the page. Note: If Slack is not available or an alternative means of communication is required, PagerDuty has cell phone numbers of GitLab team members involved in the on-call process. GitLab also sets up a Zoom channel (Slack Down!) as backup communication channel.
- If the Security Engineer On-Call is unresponsive, attempt to contact other Engineers to take on the page. Prioritize based on timezone and region.
- In the event of a high-impact security incident to GitLab, the Security Manager On-Call will be engaged to assist with cross-team/department coordination.

Triage Engineer

During On-Call shifts the Security Engineer On-Call is the Triage DRI and has these core responsibilities:

1. Acknowledge and triage pages; delegate
1. Monitor and triage incidents; delegate
1. Monitor and triage alerts; delegate alerts that are for their own activity; delegate resulting incidents
1. Continue critical assignments as determined by management

Weekday only:

1. Monitor and respond to security related slack channels; delegate
1. Improve on-call and incident handling processes - document on-call related problems and improvement opportunities by opening issues and any necessary support tickets (e.g. Devo support).
1. Continue ongoing work where possible

Triage Engineer Responsibility to delegate

Delegation allows the team to spread the workload across the global team while maintaining adequate coverage and response times. It also minimizes the risk of one single person having to handle spikes in incident volume.

- On-Call Engineers must delegate incidents to other Engineers after performing initial triage.
- Delegation guidelines:
 - Incidents classified with either S1, P1, S2, or P2 should only be delegated to team members available in the current sunny region.
 - All other incidents can be assigned to any team member not out of office in any region.

- Sometimes, team members are working on a high-priority, time sensitive assignment and are temporarily unavailable for delegation. These situations should be communicated to the team transparently. If those team members inadvertently receive delegated incidents, they should work with their manager for assistance in incident reassignment.
- The On-Call Engineer is the last resort if no other team member is available to work on high-urgency incidents.

Triage Process

Incident Triage

Incidents are triaged by following the checklist posted by the TriageBot.

Alert Triage

Alert triage is a continuous task on weekdays and a task that should be done at least once a day on weekends.

Any alert can be converted to an incident and assigned to another team member following the triage delegation guidelines above.

Incident Ownership

Whoever is assigned to the incident after the initial triage is the person responsible for incident resolution. Use the assignee field in the GitLab incident to identify the responsible person. In case of a priority 1 incident, it should have one assignee per region. In these cases, the work should continue around the globe until the incident is contained and the priority lowered. In that case, the incident should go back to only having one assignee and DRI. When multiple people work on one incident, work is divided into tasks with their corresponding assignees.

Ownership of an incident means being the person responsible for:

- Ensuring continued progression to a contained and resolved state
- Maintaining ongoing 24/7 coverage of high-severity incidents
- Accurate and timely issue tracking and communication with stakeholders
- Adequate documentation and communications

Being the responsible person does not imply being the sole person to perform incident tasks. Team members from all departments can be called upon to assist in incident resolution, and these requests should be documented as a task or related issue.

Exceptions

Exceptions to this procedure will be tracked as per the Information Security Policy Exception Management Process.

SECTION: security/security-operations/red-team/_index.md

What Are You Looking For?

- "Is this the Red Team? / Is the activity I'm seeing part of a Red Team operation?" Did you find something potentially malicious and you think it might be us? :pointright: Is this the Red Team?
- "I have some attacker related ideas that could be quite interesting!" :pointright: Contact us
- "Do you do penetration testing?" :pointright: No. :smile: We don't do vulnerability enumeration. See Services we offer
- "I want to upskill in Red Teaming" :pointright: See resources, Club Red
- "Can you hack me?" :pointright: GitLab team members can sign up to our laptop opt-in programme
- "Can you help us with an incident? Does this look like hacker stuff in these logs?" :pointright: Contact us
- "Can you help find bugs in GitLab the product?" :pointright: You're better off asking Security Research

Contact Us

- GitLab team members:
 - Come chat with us on Slack in sdsecurityredteam or via Direct Message.
 - Open an issue in our internal issue tracker
- General public:
 - Open an issue in one of our public projects

General Information

Vision

To continuously strengthen GitLab's defenses against evolving real-world threats while leading the offensive security industry through transparency, collaboration, and technical excellence.

Mission

We emulate real-world adversaries to increase GitLab's threat resilience (our ability to prevent, detect, and respond to cyber attacks). By conducting well-planned exercises in a safe and controlled manner, we give teams across GitLab the chance to practice and improve their defensive capabilities.

We achieve this by:

- Working with Threat Intelligence to identify and emulate the most relevant threats
- Partnering with teams across GitLab to translate security observations into actionable improvements
- Collaborating openly with industry peers to share security practices

While our work requires us to think and act like adversaries, we remain firmly grounded in the GitLab values. We intentionally bring transparency and collaboration to Red Teaming, conducting all operations thoughtfully and safely to achieve the best possible results for our organization.

The Team

The Red Team is part of the Security Operations department. See the GitLab organizational chart in Workday and meet our team members.

Further details about Red Team roles can be found in the job family description.

Services We Offer

In everything we do, we follow our rules of engagement (RoE).

Stealth Operations

Our primary service. Stealth operations are typically 3-9 months in length and are focused on emulation of relevant threats to GitLab. We work closely with Threat Intelligence to identify an adversary that might pose a security risk to GitLab. Using tools we've developed similar to those of the adversary, we carry out the attack on GitLab's systems. We use stealth, testing the defenses realistically and without introducing unnecessary risk. [Read more...](#)

Opportunistic Attacks

These are short (a few days), more spontaneous attacks against GitLab systems when a potential initial access vector comes to our attention. Depending on what we find, we can either raise an incident ourselves, or continue as attackers would until discovery. We're looking to formalise this to a framework for discovering initial access more efficiently and iteratively. [Read more...](#)

Purple Teaming

Purple Team represents a collaborative exercise between the Red Team and Blue Team (our defensive teams, usually SIRT or Signals Engineering). These can be:

- flash operations, which are 1-2 week exercises triggered by Threat Intelligence Flash Reports to rapidly test our defenses against emerging threats, or
- longer-term collaborations such as Atomic Testing

[Read more...](#)

Research

Understanding emerging technologies and tools helps us anticipate adversaries' evolving tactics. It also means we can more effectively emulate those attackers.

Formalized operations and opportunistic attacks both require extensive research, and we factor that in when planning these activities. Outside of that context, the Red Team may conduct research with an intent to provide helpful information to others in the security industry and the wider GitLab community.

Examples include:

- social engineering techniques
- stealth and defense evasion techniques
- AI for offensive security
- tools such as enumeration/scanning tools, command and control (C2) frameworks
- malware found in the wild

We publish tech notes that summarize some of our research as part of responsible disclosure. Explore our list of tech notes, blogs and other research and public Git repository of some past tooling and techniques.

How We Operate

The Red Team operates under a predefined set of rules of engagement. These rules provide guidelines for determining scope, the values we employ during our engagements, how we collaborate as a security team, and how we escalate vulnerabilities and exploits we discover during those engagements.

We also track results via metrics which are available internally, including MITRE heatmap to ensure coverage of relevant attacker activity.

[Read more...](#)

Additional Resources/Further Reading

GitLab Resources

- Public Red Team repository
 - Contains tools, scripts/proofs of concept and tech notes that have been made public
- Internal Red Team repository (Available for GitLab team members only)
- Our list of tech notes, blogs and other research
- Our public Git repository of some past tooling and techniques

External Resources

- Red Team Development and Operations: An excellent book by Joe Vest and James Tubberville.
- Purple Teaming Execution Framework: Another great resource written by Scythe.
- MITRE ATT&CK: Getting Started: A collection of resources related to the ATT&CK framework, which is used as the foundation for much of our work.

Is This The Red Team?

GitLab team members: See something potentially suspicious and want to check in with us? We won't answer this question. Read more about why in [Is This the Red Team?](#)

All suspicious activity should be treated as potentially malicious and acted upon accordingly.

SECTION: security/security-operations/red-team/stealth-operations.md

Stealth operations is our most significant offering, providing GitLab an opportunity to practice responding to real-world attacks. We use stealth operations to emulate threats most likely to target GitLab, our platform, and our customers. This focused approach sharpens our defenses and keeps us ahead of potential attacks.

Who is involved?

During a stealth operation, only a small group of GitLab team members are aware of the details. We call these people "trusted participants", and they help keep operations safe and productive.

How long do stealth operations last?

Stealth operations can vary in format and length. Some follow a more defined cycle with clear start and end dates, typically spanning 3-6 months. These operations often conclude when a significant detection event occurs, allowing us to evaluate the full response process. Other operations are continuous, designed to emulate persistent threats. In these cases, if we're detected, we regroup, adapt our tactics, and continue pursuing our objectives - just as real adversaries would.

What happens during a stealth operation?

Stealth operations follow special rules. Examples of techniques we might use or those we specifically avoid are available in Stealth Operation Techniques.

What happens after?

We release a report which summarises the entire operation and our recommendations, suitable for a broad audience.

SECTION: security/security-operations/red-team/how-we-operate/_index.md

General operation guidelines

We aim to safely and effectively conduct operations in order to emulate relevant adversaries to GitLab.

- We work with the Threat Intelligence team to identify the most relevant threats to emulate.
- We conduct operations ethically and responsibly without causing harm to GitLab or its team members.
- We maintain a good trust relationship with the rest of the Security division and the wider GitLab company.

It's important to us to intentionally and enthusiastically collaborate with the rest of the company, to balance out the semi-private, sometimes hidden nature of our work.

Win together: our goal is to improve security at GitLab, and that's the same goal our defensive teams have. We "win" when GitLab wins and security is improved - whether that's by us doing super 1337 hax or by SIRT stopping us in our tracks. We're not trying to establish "dominance"

over defensive teams, we partner with them.

1. Initial access vector

There are several ways we emulate initial access:

- Research. We may conduct operations specifically looking for initial access vectors to exploit. These require substantial time and resources, so we ensure the investment is justified by the potential for security improvements and learning. For example, the 2024 Okta bypass was researched by our team before being responsibly disclosed to Okta.
- Opportunistic. Red Team members can also hunt for ways to "break in" to GitLab at any time in the context of an Opportunistic Attack. This allows us to draw attention to any discoveries and GitLab can quickly remediate. Successful intrusions can then be re-used in future stealth operations as proof of a realistic initial access vector.
- Collaborative. Club Red allows team members to collaborate with us to develop an initial access plan. We can leverage their domain knowledge for a greater overall security result for GitLab.
- Assumed Breach. Sometimes we create a scenario where we gain initial access to GitLab systems through a trusted insider. This is done in a realistic manner, leaving indicators of compromise (IoCs) that reflect an actual breach. From there, we focus on post-exploitation tactics and techniques such as establishing persistence and elevating privileges.

2. Operation execution

All operations follow our rules of engagement. We try to leave realistic indicators of compromise to simulate a realistic attack by the chosen emulated attacker.

3. Internal disclosure or discovery

A given operation will continue until we are detected or until we disclose internally. Depending on what we find during an operation (for example, if we discover a significant security risk), we may disclose early to mitigate risk.

Social resolution

Sometimes our operations involve attacking infrastructure set up by a certain team, socially engineering individual team members, or compromising a system due to misconfiguration set up by a team member.

In any retrospective, we always aim to focus on improvements rather than assign blame.

If social engineering is involved, we must be careful to ensure that the individuals involved in the exercise feel well supported and not blamed.

We offer to meet with anyone involved in our social engineering efforts to reassure them and thank them for being part of our operation.

We never want anyone to feel like they did something wrong, since our operations test processes, not individuals.

4. Report and recommendations for security improvements

We then release a report summarizing the operation and our recommendations for improving security posture. We create issues using the issue template, apply the relevant labels, and use this to track metrics. We then provide our tools and techniques to the Blue Team so they can create relevant detections.

We often work with Signals Engineering and Security Incident Response Team (SIRT) to review our findings, attack steps and review detections and alerts.

Reporting

All operations end with a final report. We use a publicly shared issue template.

Security risks affect everyone, and it is essential to make our reports approachable and consumable to a broad audience. Our goal is to ensure that anyone in the company can understand the reports, even if they don't have a technical or security background, so we strive to use simple language.

If we feel it's needed, we also provide a short (five minutes or less) video summary.

For stealth or higher-visibility operations, it's beneficial to share the story with the entire company. In that case, we post the following to the Slack channel `whats-happening-at-gitlab` and cross-post it in security:

- A very short summary of the operation, including the video overview if there is one
- A link to the final report
- A link to the retrospective issue to solicit questions and feedback
- A disclaimer to keep the information SAFE.
- A tagged list of individual operation participants as well as the Security Incident Response Team (SIRT) as a whole for awareness since not all team members participate in an operation each time.

By doing this, we help foster a culture of security awareness throughout the organization and ensure that everyone can benefit from our work.

Red Team Maturity Model

We use a custom maturity model to measure our progress and help guide our decisions. This is loosely based on the Capabilities Maturity Model (CMM). Our model (available internally only) contains five stages of maturity, each with very specific behaviors we strive to demonstrate and states we hope to achieve.

Red Team Metrics

Adoption Rate

A successful Red Team program strengthens an organization's security through recommendations that are accepted and implemented by the organization. We track the lifecycle of these recommendations through to implementation using GitLab.com, calling this metric our "Adoption Rate."

Recommendations start as GitLab.com issues in the project closest to the team that can address them. We classify recommendations using labels:

- Detections & Alerts (RTRec::Detection)
- Security Controls (RTRec::Control)
- Processes and Procedures (RTRec::Process)

We track what happens after a recommendation is delivered by adding a secondary label to the issue with its current status or final outcome:

- Under review (RecOutcome::UnderReview)
- Accepted and actively being worked on (RecOutcome::InProgress)
- Accepted but backlogged (RecOutcome::Backlogged)
- Accepted but blocked (RecOutcome::Blocked)
- Fully adopted and closed (RecOutcome::Adopted)
- Partially adopted and closed (RecOutcome::PartiallyAdopted)
- Not adopted and closed (RecOutcome::NotAdopted)

An outcome label is added to the issue within one week of delivering the recommendation. A scheduled CI pipeline checks issues with classification labels to ensure they have an outcome.

MITRE ATT&CK Mapping

MITRE ATT&CK is a framework for classifying and describing cyber attacks. We use ATT&CK extensively, because it helps us to align our operations to realistic threats and to speak a common language across security groups.

We use a combination of GitLab CI pipelines and GitLab Pages to build and host two reporting tools from MITRE:

- MITRE ATT&CK Flow
 - Our internal automation project
 - Our internal build
- MITRE ATT&CK Navigator
 - Our internal automation project
 - Our internal build
 - Public blog describing the project

For each completed operation, we build a flow chart to visualize the attack path and indicators of compromise. This chart can be exported as a STIX 2.1 compliant JSON file, meaning it is machine-readable and can be imported into other tools for analysis.

That same ATT&CK Flow file is imported into our ATT&CK Navigator project, which generates a heatmap visualizing our coverage across the ATT&CK matrix. We maintain a single heatmap for each operation, as well as a combined heatmap for all previous operations.

This is a great way to visualize the types of attack techniques we've emulated and helps us understand areas to focus in future operations.

Is this the Red Team?

Why we don't answer this question

The goal of a Red Team operation is often to test our policies and procedures when reacting to an actual threat. This includes identifying suspicious activity and following the appropriate runbook to investigate and respond to that threat.

If any team member, at any time, could simply ask "Hey, this looks suspicious. Is this our Red Team?" then this opportunity would be lost. Instead, all suspicious activity should be treated as potentially malicious and acted upon accordingly.

We have private Slack channels in place where designated team members can ask the Red Team if a certain activity belongs to them. This helps us to provide realistic opportunities to practice detection and response without escalating too far. For example, we would not want an emulated attack to affect production operations or escalate to third parties.

Managers at GitLab can also submit a "Red Team Disclosure Request" at any time. If the request contains evidence related to an ongoing Red Team operation, we will discuss next steps in the Slack channels mentioned above.

You can read more about this process in the "Requests for Disclosure" section of our rules of engagement.

How the Red Team will respond to this question

If the Red Team is ever asked "Is this you?" by someone other than the designated team members mentioned above, they will respond with the following text:

> Thanks for your vigilance! Any suspicious activity should be treated as potentially malicious. If you'd like to contact security, you can follow the process [here](#).

>

> Red Team operations provide an opportunity to practice detecting and responding to real-world attacks, and revealing an operation early might mean we miss out on that opportunity. Because of this, we have a policy to neither confirm nor deny whether an activity belongs to us. You can read more about this policy [here](#): [\[.\]\(/handbook/security/security-operations/red-team/how-we-operate/.is-this-the-red-team\)](#).

How others should respond to this question

Because we want to treat all activity as potentially malicious, anyone else receiving this question should also use a consistent response. Feel free to use your own words. The following can be a guide:

> We want to treat any suspicious activity as potentially malicious. Let's continue following our normal procedures to report and investigate this. Any Red Team operation has controls in place to keep things from escalating too far. You can read more about this [here](#): [\[.\]\(/handbook/security/security-operations/red-team/how-we-operate/.is-this-the-red-team\)](#).

If the person receiving this question happens to be a Security Director or a trusted participant in an ongoing stealth operation, they can then use established channels to communicate with the Red Team.

SECTION: security/security-operations/red-team/how-we-operate/rules-of-engagement.md

This page outlines the general rules that apply to all work conducted by the Red Team. Individual operations may include additional rules defined during planning stages.

Please refer to our general handbook page to learn more about our team and what we do.

Systems in Scope

General Systems

Some systems require special approval from GitLab legal before accessing during any type of Red Team operation. GitLab team members can view that list [here](#).

Third-party systems that are used by GitLab for official business purposes are also considered in scope, but these often require permission from the system owners. This permission will be obtained when necessary.

All other systems managed by GitLab are in scope for all types of Red Team operations.

Team members can request that a system be excluded from our scope by opening an issue [here](#).

Company-Issued Laptops

By default, company-issued laptops are not in scope unless authorized by the team member who uses the laptop.

GitLab team members can opt-in to having their laptops included in the scope of future operations. We encourage participation in this program, as it allows us to emulate the most realistic threats and to propose improvements to prevent actual attacks from succeeding.

If an in-scope laptop is compromised, the Red Team may run additional commands on the laptop to establish persistence, extract company-related credentials, and move laterally through GitLab's environment. We respect the privacy of our team members and will not attempt to access webcams/microphones, monitor home networks, or attempt to access personal information.

Everything the Red Team does on a compromised laptop will be logged and auditable.

We appreciate our opt-in participants and we know that breaches can happen to anyone. Our goal is to continuously improve our ability to prevent and detect security incidents, not to place any blame on individuals. A compromised opt-in laptop will not lead to any disciplinary measures or negatively impact a talent assessment.

If a laptop is compromised during an operation, the team member will be notified when the operation is complete. Their name will not appear in the operation's final report, but they may be tagged in issues related to investigation and remediation of the compromise.

Team members can opt in to this program using this [Google form](#). Those who change their minds

can use the same form to opt back out. If a team member decides to opt out after their laptop has already been involved in an emulated compromise, the Red Team will delete any data that may have been extracted from their laptop. This is an unlikely scenario, but we want to take extra steps to ensure everyone's privacy.

Please check with your manager to ensure they have no concerns with your participation prior to opting in.

Stealth Operations

Stealth Operations require careful planning. During the logistics phase, we propose objectives and outline the threats we'll emulate, seeking approvals in this template.

The sections below contain general rules that apply to all stealth operations.

Prior Approval

- Approval is required from at least one layer of SIRT leadership. This starts at the SIRT managers and goes up to the CISO.
- Approval is required from the Senior Manager responsible for the Red Team.
- Approval is required from everyone defined as a "trusted participant" in the logistics phase.
- Approval is provided as part of the logistics phase of every stealth operation. The scope of the approval is included as part of the logistics issue where approval is recorded.

Requests for Disclosure

- Active stealth operations will have a dedicated Slack channel where all trusted participants are invited.
- The private is-this-the-redteam channel is always available for Security Directors and above to inquire about Red Team activities.
- Managers and above can submit an issue using this template to request disclosure of Red Team activities. This template contains details on how these issues are handled.
- If team members ask whether a specific activity or IoC belongs to the Red Team outside of these designated channels, we will follow the process documented in "Is This The Red Team?".
- If asked in one of the designated Slack channels, the following will happen:
 - Any ongoing stealth activities will be paused until a definitive answer can be provided.
 - If it was not the Red Team, the requestor can still ask that all activities remain paused until the issue is resolved.
 - If it was the Red Team performing a stealth operation, there will be a documented plan (from the "logistics" phase) on what happens next. Some examples are:
 - The operation continues in stealth, with the trusted participant helping to ensure any escalation is in-line with the operation goals.
 - The operation is disclosed internally and continues in the open to further validate security controls and detection mechanisms. Incident response is discontinued for known Red Team activities.
 - The entire operation is not disclosed internally, but the trusted participant informs the team that a specific activity or IoC belongs to the Red Team. The systems involved are considered "virtually contained" and the Red Team relinquishes any access to those systems.
 - The operation ceases completely.
- We should always prioritize the security of the organization and the well-being of our team

members. There may be times when we need to make a judgement call and answer these questions quickly, outside of the official channels.

Engaging with SIRT

At GitLab, our Red Team and Blue Team have a long history of working collaboratively towards a common goal. Despite playing roles that are adversarial in nature, we want to maintain a very high level of trust and respect between the teams.

We define some specific rules below, but in general we want to remember to always be kind and compassionate in our work. If something feels like it conflicts with GitLab's values, we need to re-evaluate exactly what we are doing and why we are doing it.

- The Red Team will not intentionally distract or mislead SIRT using official company communication channels under the team's legitimate user accounts.
- The Red Team will make an effort to avoid triggering on-call incidents outside of normal business hours. In general, this means not conducting attack techniques during hours when it is a weekend or holiday globally.
- SIRT will not proactively monitor specific Red Team resources. If any alerts or standard incident response procedures lead to these resources, of course they may be investigated by SIRT. The intent of this rule is to avoid prematurely uncovering a Red Team operation in an unrealistic manner. These resources include:
 - Red Team members' laptops
 - Red Team's group project inside Google Cloud
 - Red Team's private projects on GitLab.com
 - Red Team's private Slack channels and direct messages

General Safety Guidelines

The sections below outline general rules all Red Team operations.

Team Member Privacy

As security professionals, we aim to be ethical in every engagement while maintaining the spirit of mimicking real-world attacks. We respect the privacy of the employees at GitLab and follow the guidelines mentioned in the Employee Privacy Policy during our engagements.

Critical Vulnerabilities and Exploits

The Red Team may discover and exploit vulnerabilities during an engagement. These will not always be reported immediately to SIRT, as we want to provide a realistic opportunity to detect and respond to that exploitation.

If a vulnerability is exposed that meets the following criteria, we will document the issue and follow the process to engage SIRT immediately:

- Vulnerability is exposed publicly
- Vulnerability is realistically exploitable
- Exploitation of the vulnerability would impact business operations and/or expose sensitive data

Emergencies

Red Team operations are carefully planned to avoid impacting production systems and customer activities. However, accidents may occur and production systems may respond in an unpredictable manner.

If we ever suspect an impact to production, we will do the following:

1. Suspend any activities related to the impact
1. Immediately inform everyone defined as a "trusted participant" for the operation
1. If a security incident is required, engage SIRT
1. If an infrastructure incident is required, engage the on-call SRE
1. Perform a proper root cause analysis following resolution of the incident

Common Techniques

This section describes techniques that are commonly used by the Red Team. These lists are not all inclusive, but are meant to provide team members reasonable expectations of things the Red Team may or may not do.

If any team members have questions or concerns about these operations, please contact us in the gsecurityredteam Slack channel.

Opportunistic Attack Techniques

Techniques used in opportunistic attacks include:

- Port scanning.
- Web crawling and scraping.
- Manual or automated vulnerability scanning.
- Manual or automated exploit attempts on vulnerable software or infrastructure.
- Manual or automated attempts at abusing misconfigurations.
- Manually and programmatically querying the GitLab API.
- Accessing and cloning any public projects, issues, snippets, etc. on www.gitlab.com.
- Accessing other data intended to be open to the public, such as logging platforms.
- Attempting to log in to any publicly-exposed administrative interface with common and default credentials.
- Attempting to validate credential data such as GCP service accounts, SSH keys, and API keys found in public locations.
- Gathering open-source intelligence (OSINT) from any source, including those not managed by GitLab.

Stealth Operation Techniques

During a stealth operation, the Red Team may:

- Begin an operation assuming a breach has occurred, meaning the team will have some level of access to resources that are not exposed publicly.
- Attempt to gain access to team member accounts on any GitLab-managed platform (GitLab.com, Google, Slack, Zoom, etc).

- Attempt to gain access to in-scope laptops using techniques like drive-by attacks, spearphishing with attachments, and supply chain compromises.
- Search for credential data on any compromised system. This includes environment variables, configuration files, shell history, etc.
- Authenticate as any team member using credential data discovered on any GitLab-managed resource (passwords, access tokens, SSH keys, etc).
- Attempt to gain access to any resource in GitLab's cloud environments, and use those resources to escalate privileges and move laterally in the cloud.
- Exploit vulnerabilities and abuse insecure configurations in any system owned and managed by GitLab.
- Conduct social engineering using techniques like:
 - Creating fake identities and using them to engage with team members o